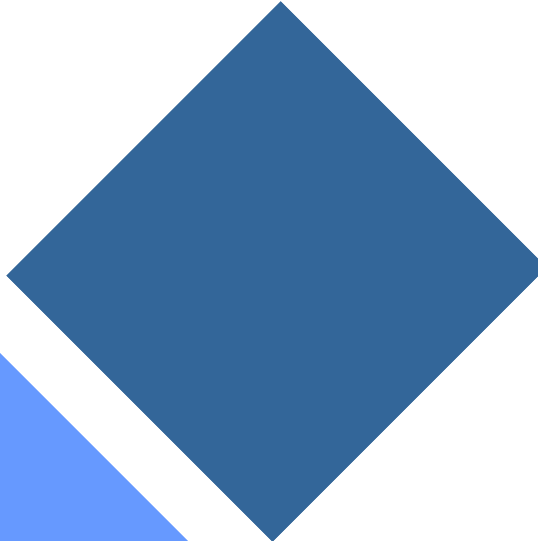


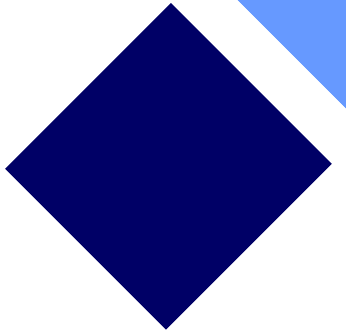
SP2: Ecosistema Spark

Isabel Pelaya Galindo Ibáñez
Esther Ibáñez Mingorance
José Luis López García
Juan Rafael Madolell Usero





Indice



1. DRIVER

2. LA SPARKSESSION

3. EXECUTORS

4. RAM Y CORE

5. DATAFRAME

6. FLUJO

7. SOLUCIÓN

8. PROBLEMAS

1. DRIVER

El **Driver** es el proceso que ejecuta la aplicación Spark.

Se encarga de:

- Crear la SparkSession
- Construir el plan lógico de ejecución
- Dividir el trabajo en Jobs, Stages y Tasks
- Coordinar la comunicación con el clúster
- Recoger y consolidar resultados

El Driver no procesa grandes volúmenes de datos, su función es **coordinar**.

2. LA SPARKSESSION

La **SparkSession** representa la conexión entre la aplicación y el clúster.

¿Cómo funciona?

- Configurar la aplicación
- Definir el Master
- Establecer parámetros de ejecución
- Crear DataFrames y ejecutar consultas

Sin SparkSession no existe contexto distribuido.

3. EXECUTORS

Los **Executors** son procesos que:

- Ejecutan las tareas asignadas por el Driver
- Procesan datos en paralelo
- Mantienen datos en memoria
- Devuelven resultados parciales

Componentes:

- RAM
- CPU

El número de Executors y sus recursos determinan la **capacidad** del clúster.

4. RAM Y CORE

Spark distribuye el trabajo según los recursos disponibles.

- La memoria (RAM) permite almacenar datos en ejecución.
- Los núcleos de CPU determinan el grado de paralelismo.

Más **CORE** = Más tareas simultáneas

Más **RAM** = Menos inscripción en disco

5. DATAFRAME

Un **DataFrame** es una estructura de datos distribuida organizada en filas y columnas.

Características:

- Distribuido entre los Executors
- Inmutable
- Similar conceptualmente a una tabla SQL

El DataFrame es la abstracción más utilizada en Spark para trabajar con datos estructurados.

DataFrame e Inmutabilidad

Los DataFrames son **inmutables**.

Cuando aplicamos una transformación:

- No se modifica el DataFrame original
- Se crea un nuevo DataFrame
- Se registra la operación en el plan lógico

Ventajas:

- Seguridad en ejecución paralela
- Tolerancia a fallos
- Optimización automática

6. FLUJO

- 1.El Driver crea la SparkSession.
- 2.Se define un DataFrame.
- 3.Se aplican transformaciones.
- 4.Spark construye el DAG. (Rango de 0 a 10.000)
- 5.Una acción activa la ejecución.
- 6.Los Executors procesan las particiones.
- 7.El resultado vuelve al Driver.

7. SOLUCIÓN

ArchivoEditarVerHistorialMarcadoresPerfilesHerramientasAyuda

Ver evaluación

Application: SparkStressTest

←→↺🏠

🔒📄http://localhost:8080/app/?appId=app-20260303155548-0000

📄🔍🌟

⬇️👤Iniciar sesión🔖⚙️☰

APACHE

spark

4.0.1

Application: SparkStressTest

ID: app-20260303155548-0000

Name: SparkStressTest

User: spark

Cores: Unlimited (2 granted)

Executor Limit: Unlimited (2 granted)

Executor Memory - Default Resource Profile: 512.0 MiB

Executor Resources - Default Resource Profile:

Submit Date: 2026/03/03 15:55:48

Duration: 5.9 min

State: RUNNING

Application Detail UI

▼ Executor Summary (2)

ExecutorID	Worker	Cores	Memory	Resource Profile Id	Resources	State	Logs
0	worker-20260303155322-172.22.0.4-35803	1	512	0		RUNNING	stdout stderr
1	worker-20260303155322-172.22.0.3-42713	1	512	0		RUNNING	stdout stderr

7. SOLUCIÓN

Archivo Editar Ver Historial Marcadores Perfiles Herramientas Ayuda

Ver evaluación x Application: SparkStressTest x +

Problems Output Debug Console **Terminal** Ports

```
26/03/03 15:53:24 INFO Worker: Worker cleanup enabled; old application directories will be deleted in: /opt/spark/work
26/03/03 15:55:48 INFO Worker: Asked to launch executor app-20260303155548-0000/1 for SparkStressTest
26/03/03 15:55:48 INFO SecurityManager: Changing view acls to: spark
26/03/03 15:55:48 INFO SecurityManager: Changing modify acls to: spark
26/03/03 15:55:48 INFO SecurityManager: Changing view acls groups to: spark
26/03/03 15:55:48 INFO SecurityManager: Changing modify acls groups to: spark
26/03/03 15:55:48 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: spark gr
roups with view permissions: EMPTY; users with modify permissions: spark; groups with modify permissions: EMPTY; RPC SSL disabled
26/03/03 15:55:48 INFO ExecutorRunner: Launch command: "/opt/java/openjdk/bin/java" "-cp" "/opt/spark/conf:/opt/spark/jars/slf4j-api-2.0.
16.jar:/opt/spark/jars/*" "-Xmx512M" "-Dspark.driver.port=35825" "-Djava.net.preferIPv6Addresses=false" "-XX:+IgnoreUnrecognizedVMOptions"
"--add-modules=jdk.incubator.vector" "--add-opens=java.base/java.lang=ALL-UNNAMED" "--add-opens=java.base/java.lang.invoke=ALL-UNNAMED"
"--add-opens=java.base/java.lang.reflect=ALL-UNNAMED" "--add-opens=java.base/java.io=ALL-UNNAMED" "--add-opens=java.base/java.net=ALL-UN
NAMED" "--add-opens=java.base/java.nio=ALL-UNNAMED" "--add-opens=java.base/java.util=ALL-UNNAMED" "--add-opens=java.base/java.util.concur
rent=ALL-UNNAMED" "--add-opens=java.base/java.util.concurrent.atomic=ALL-UNNAMED" "--add-opens=java.base/jdk.internal.ref=ALL-UNNAMED" "-
--add-opens=java.base/sun.nio.ch=ALL-UNNAMED" "--add-opens=java.base/sun.nio.cs=ALL-UNNAMED" "--add-opens=java.base/sun.security.action=AL
L-UNNAMED" "--add-opens=java.base/sun.util.calendar=ALL-UNNAMED" "--add-opens=java.security.jgss/sun.security.krb5=ALL-UNNAMED" "-Djdk.re
flect.useDirectMethodHandle=false" "-Dio.netty.tryReflectionSetAccessible=true" "org.apache.spark.executor.CoarseGrainedExecutorBackend"
"--driver-url" "spark://CoarseGrainedScheduler@spark-master:35825" "--executor-id" "1" "--hostname" "172.22.0.3" "--cores" "1" "--app-id"
"app-20260303155548-0000" "--worker-url" "spark://Worker@172.22.0.3:42713" "--resourceProfileId" "0"
```

ID: app-20260303155548-0000
Name: SparkStressTest
User: spark
Cores: Unlim
Executor Lin
Executor Me
Executor Res
Submit Date
Duration: 5.9
State: RUNN
Application

▼ Executors

ExecutorID
0
1

8. PROBLEMAS

- **Dependencias Locales**

Al intentar ejecutar el script directamente en Windows con python spark_stress_test.py, dio error porque PySpark necesita tener Java (JAVA_HOME) instalado en el sistema local para que el Driver funcione

SOL: En lugar de instalar Java en Windows, decidimos ejecutar el script dentro del contenedor Docker del Master, que ya tiene todo el entorno de Spark y Java preparado

8. PROBLEMAS

- **Aislamiento de Archivos**

Los contenedores Docker no podían ver nuestro archivo.

SOL: Tuvimos que modificar el docker-compose.yml para añadir un volumen (- ./opt/spark/work-dir). Esto crea un túnel entre la carpeta de Windows y el interior de los contenedores.

8. PROBLEMAS

- **Nombres y Comandos**

El primer intento de ejecución en Docker falló porque usamos el comando python, pero el contenedor solo reconocía python3.

SOL: Usamos python3 y, más tarde, spark-submit.

**Muchas
gracias**

